Oct 05, 2026

Virtual Memory MCP Service: Build, Validate, and Release Protocol

Created Oct 05, 2026

Last Modified Oct 05, 2026

Experiment ID
W0_

Number of steps
29 / 29 completed

Abstract

End-to-end delivery protocol for designing, implementing, testing, approving, and releasing a governed MCP virtual-memory service.

Safety warnings

Memory can influence future agent behavior. Do not enable autonomous approval, cross-tenant retrieval, unrestricted database access, or storage of credentials. Treat all external content as untrusted data until it passes the documented validation and approval workflow.

1. Define scope and accountable roles

1	Name the product owner, service owner, security owner, data-governance owner, retrieval owner, and release approver. Record each person's authority for requirements, data classification, policy exceptions, deployment, and rollback.	Done October 5th 2026 04:06:40 am
2	Write the MVP boundary. Include scoped retrieval, lifecycle states, provenance, candidate write proposals, approval or	Done October 5th 2026 04:06:39 am

rejection, revocation, audit events, tenant isolation, and hybrid retrieval. Explicitly exclude unrestricted autonomous writes, cross-tenant search, secret storage, and broad database access.

3 Define the protected assets and failure conditions: private memory content, tenant boundaries, approval integrity, source provenance, audit integrity, retrieval correctness, and secret references.

Done
October 5th 2026
04:06:38 am

2. Establish policy and data model

4 Define memory classes: working, episodic, semantic, procedural, relationship, and audit. For every class, document permitted sources, default retention, retrieval rule, approval requirement, sensitivity expectations, and deletion behavior.

Done
October 5th 2026
04:06:36 am

5 Implement lifecycle states: candidate, validated, approved, active, superseded, expired, revoked, and deleted. Define permitted state transitions and the identity or policy authorized to perform each transition.

Done
October 5th 2026
04:06:34 am

6 Define the required memory metadata: stable ID, tenant, content or protected content reference, type, lifecycle state, trust level, source identifier and locator, source digest, collector, timestamps, retention, scope, sensitivity, creator, validator, approver, version links, and audit correlation ID.

Done
October 5th 2026
04:06:33 am

7 Define a default-deny authorization matrix for user, project, department, organization, agent role, workflow, memory class, and action. Ensure that tenant filtering is mandatory before lexical, vector, or graph retrieval.

Done
October 5th 2026
04:06:32 am

3. Build the secure MCP surface

8 Expose only narrow operations for search, get, explain, propose_write, approve_write, reject_write, revoke, and audit. Validate each request and response against a strict schema. Do not expose a generic database query or an unrestricted write operation.

Done
October 5th 2026
04:06:30 am

9 Require an authenticated principal, tenant, agent identity, job or task identifier, requested purpose, and correlation ID on every request. Reject requests missing required context.

Done
October 5th 2026
04:06:29 am

10 Implement policy enforcement at the gateway and again within the memory service. Return policy-safe denials that do not disclose whether protected memory exists.

Done
October 5th 2026
04:06:27 am

11 Store references to a secret vault when a workflow requires credentials. Reject attempts to write API keys, tokens, passwords, session cookies, or raw secrets into memory.

Done
October 5th 2026
04:06:26 am

4. Implement ingestion and governance

12 On ingestion, capture source identity and locator, calculate a content digest, classify sensitivity, identify tenant ownership, and retain the original source reference where policy permits.

Done
October 5th 2026
04:06:24 am

13 Treat web pages, emails, uploaded documents, retrieved resources, and tool outputs as untrusted data. Detect and flag instruction-like content, attempts to alter policy, or content requesting data exfiltration.

Done
October 5th 2026
04:06:22 am

14 Create all new writes as candidate records. Run validation for provenance, duplication, scope, retention, classification, and memory class before any approval decision.

Done
October 5th 2026
04:06:21 am

15 Require a human or defined policy authority to approve high-impact writes, including policies, permissions, legal guidance, financial facts, cross-user memories, and access-scope changes. Record decision rationale.

Done
October 5th 2026
04:06:19 am

5. Implement retrieval and evidence packets

16 Apply authorization, tenant, lifecycle, retention, trust, time, type, and sensitivity filters before candidate retrieval. Exclude deleted, expired, revoked, superseded where appropriate, and unauthorized records.

Done
October 5th 2026
04:06:18 am

17 Run lexical and vector candidate retrieval. Expand graph neighbors only when task relevance and policy justify it. Re-rank using relevance, recency, source quality, trust, version authority, and scope fit.

Done
October 5th 2026
04:06:15 am

18 Return a capped evidence packet rather than an archive. Include memory ID, authorized content or excerpt, type, trust and lifecycle state, source label, timestamps, expiration, version reference, and explanation reference.

Done
October 5th 2026
04:06:14 am

19	Record the query context, policy decision, result IDs, response size, and correlation ID in the audit ledger. Do not log more protected content than needed for audit.	Done October 5th 2026 04:06:13 am

6. Test security and correctness

20	Create unit tests for lifecycle transitions, schema validation, scope checks, tenant isolation, retention expiry, revocation, versioning, approval rules, source digest handling, and audit-event creation.	Done October 5th 2026 04:06:10 am
21	Perform negative authorization tests. Attempt cross-tenant retrieval, access from an unauthorized agent, enumeration through errors, retrieval of revoked content, and access to memories with an expired scope. Verify all attempts fail closed.	Done October 5th 2026 04:06:08 am
22	Perform memory-poisoning tests using external content that contains hostile instructions, policy-bypass requests, misleading provenance, prompt-injection text, and invalid approval claims. Verify that content remains untrusted and cannot self-approve.	Done October 5th 2026 04:06:06 am
23	Perform retrieval-quality tests using known queries and expected evidence sets. Measure precision, missing authoritative records, stale-result rate, response size, latency, and provenance completeness.	Done October 5th 2026 04:06:05 am
24	Run a deletion and revocation test. Revoke an active memory and verify that ordinary retrieval, lexical indexes, vector indexes, and graph expansions stop returning it within the defined service objective. Verify audit preservation follows policy.	Done October 5th 2026 04:06:03 am

7. Approve release

25	Review the threat model, authorization matrix, data-class policy, retention policy, test evidence, monitoring plan, incident response path, rollback plan, and open risks. Document exceptions and their expiry dates.	Done October 5th 2026 04:06:01 am
26	Deploy first to a limited non-production or pilot scope with least-privilege agents and low-sensitivity memories. Enable enhanced monitoring for denied requests, unexpected	Done October 5th 2026 04:05:59 am

retrieval volume, candidate-write spikes, anomalous approvals, and index propagation failures.

27 Authorize production expansion only after the release approver confirms the acceptance criteria: scoped active-only retrieval, non-automatic trust promotion, complete provenance, auditable writes and retrievals, effective revocation, and demonstrated tenant isolation.

Done
October 5th 2026
04:05:55 am

8. Operate and improve

28 Review retrieval quality, policy denials, approval turnaround, retention actions, revocations, audit completeness, and security alerts at a defined cadence. Convert findings into prioritized changes with owner and due date.

Done
October 5th 2026
04:05:52 am

29 Version policies, schemas, ranking logic, and MCP operation contracts. Test compatibility before rollout and retain the policy and service version in audit records needed for reproducibility.

Done
October 5th 2026
04:05:49 am